STUDENT MANAGEMENT SYSTEM

A C++ programming-based Student Management System

Documentation Booklet

Release: v1.0
File Name: StudentManagement.cpp
© 2026 Krishna Kohli
Submitted On: 27 April 2026

INDEX

1. Abstract 3
2. Introduction 4
   - 2.1 Overview of the Project 4
   - 2.2 Purpose of the Student Management System 4
   - 2.3 Scope of the Project 5
3. Feasibility Study 6
   - 3.1 Technical Feasibility 6
   - 3.2 Operational Feasibility 6
   - 3.3 Economic Feasibility 7
4. Background Study 8
   - 4.1 Existing Student Record Systems 8
   - 4.2 Limitations in Traditional Implementations 9
5. Objectives 10
   - 5.1 Primary Objective 10
   - 5.2 Secondary Objectives 11
6. System Design & Coding 12
   - 6.1 System Architecture 12
   - 6.2 Data Structures Used 13
   - 6.3 Function Descriptions 13
7. Program Output / Sample Run 15
   - 7.1 Sample Terminal Interaction 15
8. Future Enhancements 16
   - 8.1 Adding File Handling for Persistence 16
   - 8.2 Implementing Inheritance and Polymorphism 16
   - 8.3 Database Backend Integration 17
   - 8.4 Graphical User Interface (GUI) Version 17
9. Conclusion 18
10. Submission 19

INTRODUCTION

The Student Management System is a classic administrative application in which an operator can store, retrieve, modify, and delete student academic records through a menu-driven terminal interface. This project aims to implement a simple, terminal-based Student Management System using the C++ programming language. It demonstrates the use of core Object-Oriented Programming concepts such as Classes, Encapsulation, getter and setter methods, and the C++ Standard Template Library container std::vector.

The project helps beginners understand how a real administrative system can be built from fundamental principles, without relying on complex libraries, database engines, or advanced programming techniques. It also strengthens logical thinking, algorithmic design, and the use of modular programming by separating data (the Student class) from control logic (the main menu).

2.1 Overview of the Project

This project is designed to create a straightforward and interactive Student Management System that runs in a terminal environment. The system stores student records—including Name, Roll Number, Semester, and SGPA—in an in-memory vector. The operator can add new students, view all records, search for a student by Roll Number, update existing details, and delete a record.

The program presents a numbered menu in a continuous do-while loop. The operator enters a choice, the corresponding operation is executed via a switch statement, and the menu is displayed again. Entering 0 terminates the session cleanly.

The goal of this project is not just data management but learning—specifically how classes, objects, encapsulation, vectors, and iterative menu structures work together in C++.

2.2 Purpose of the Student Management System

The main purpose of this project is to apply core C++ OOP concepts to create a fully functioning administrative application. It helps students understand:

- How to define a class with private data members and public methods
- How encapsulation protects data integrity
- How to use std::vector as a dynamic, resizable list of objects
- How to process user input safely with cin.ignore() and getline()
- How to structure a program using a do-while loop and switch statement
- How to search, update, and delete items from a vector using index-based iteration

The system also encourages problem-solving and critical thinking as the developer must handle edge cases such as missing roll numbers and input buffer clearing.

2.3 Scope of the Project

The scope of this Student Management System project is intentionally kept simple to match the knowledge level of second-semester students. The project includes:

- A dynamic list of student records stored in a std::vector
- Full CRUD operations: Add, View All, Search, Update, and Delete
- Private data members accessed via public getters and setters

- Safe input handling using cin.ignore() before getline()
- A continuous menu loop with a clean exit option
- Modular programming with class methods and a structured main() function

FEASIBILITY STUDY

A feasibility study is carried out to determine whether the development of the Student Management System in C++ is realistic, achievable, and practical within the constraints of time, knowledge, and available resources. This section evaluates the technical, operational, and economic aspects of the project to ensure that it can be developed successfully using basic programming tools and skills available to second-semester students.

3.1 Technical (Physical) Feasibility

Technical feasibility focuses on whether the project can be successfully implemented using the hardware and software tools available to the developer.

For this project, the technical requirements are very minimal. The Student Management System is implemented entirely using the C++ programming language and can run on any basic system that supports a C++ compiler such as GCC, MinGW, or MSVC. Since the program is text-based, it does not require advanced hardware like a graphics card or high processing power.

All programming concepts used—classes, encapsulation, vectors, loops, conditional statements, and simple functions—are part of the second-semester C++ OOP syllabus. No external libraries or complex frameworks are required. Hence, the project is completely technically feasible with the available tools and student knowledge.

3.2 Operational (Behavioural) Feasibility

Operational feasibility determines whether the system will function correctly and be accepted by its intended users.

The Student Management System is simple to use and requires only keyboard input from the operator. The operations are easy to understand: choose a numbered option, enter the required details, and the system responds immediately. The program displays clear prompts, formatted record lists, and success or error messages.

Since the program runs in a terminal, there is no need for training or complex setup. Anyone familiar with using a computer and typing commands can operate it easily, making the project behaviourally feasible.

3.3 Economic Feasibility

Economic feasibility examines whether the project can be completed within a reasonable cost or with no cost at all.

The Student Management System does not require any financial investment. All tools used—C++ compiler, text editor, and terminal—are free or already available on most systems. There is no need for additional hardware, software licenses, database servers, or external resources.

Because the project is simple and developed using free tools, it is economically feasible for any second-semester student.

BACKGROUND STUDY

Student record management systems have existed in various forms for decades—from paper registers and ledger books to complex relational databases used by universities today. A simplified software version, implemented in C++, offers an ideal academic exercise: it is complex enough to demonstrate real programming patterns yet simple enough to be completed within a semester.

This project aims to recreate core student data management functionality using the C++ programming language. Before building the system, it is important to study existing versions and understand the gaps or limitations that justify a new, simplified version suitable for second-semester OOP students.

4.1 Existing Student Record Systems

Several digital student management systems exist today:

- Enterprise systems — platforms like ERP-based university management tools using databases such as Oracle or MySQL, with role-based login, web interfaces, and report generation.
- Educational programming examples — many tutorials show student record logic in languages like Python, Java, or C# with features such as file storage, GUI forms, and sorting algorithms.
- Console-based C++ programs — some versions use advanced concepts such as dynamic memory allocation with new/delete, linked lists, binary file streams, or inheritance hierarchies.

Although these systems work well, they are often complex and use features well beyond the scope of a second-semester C++ OOP course. This project focuses on building a simplified version using only core concepts such as classes, encapsulation, vectors, and simple string handling.

4.2 Gaps & Limitations

While existing student management implementations are functional, they contain certain limitations when considered for beginner-level academic use:

1. Too complicated for beginners — many versions rely on advanced concepts such as file streams, pointers, templates, or inheritance not yet taught in the second semester.
2. Dependence on databases or external files — many implementations require MySQL, SQLite, or flat-file storage, increasing setup complexity significantly.
3. Lack of modularity — several beginner tutorials write all logic inside main(), making the code difficult to read, test, and maintain.
4. Not tailored for academic learning — most online versions focus on completeness and performance rather than explaining the OOP concepts behind each design decision.
5. Overuse of platform-specific commands — some versions call system("cls") or system("pause"), which do not work consistently across different operating systems.

The Student Management System developed in this project addresses these issues by providing:

- A clean, understandable two-class structure
- Modular methods inside the Student class
- Basic terminal-based CRUD operations

- No external files or database dependency
- No OS-dependent commands
- No advanced C++ concepts beyond the second-semester OOP syllabus

OBJECTIVES

The main objective of this project is to develop a simple and interactive Student Management System using the C++ programming language with a focus on Object-Oriented Programming principles. The system should function smoothly in a terminal environment and help beginners understand core OOP concepts through practical implementation. This section outlines the primary and secondary objectives that guide the development.

5.1 Primary Objective

The primary objective of this project is:

✓ **To develop a functional and user-friendly Student Management System using core C++ OOP concepts.**

This involves:

- Defining a Student class with private data members (name, rollNumber, semester, sgpa)
- Implementing public methods to add, update, display, and access student data
- Using std::vector<Student> as a dynamic, resizable in-memory database
- Handling user input correctly using cin.ignore() and getline() for names with spaces
- Allowing the operator to perform all CRUD operations through a numbered menu
- Displaying clearly formatted records with roll number, name, semester, and SGPA

The primary goal is that the system should be easy to understand, simple to run, and fully operational without any advanced programming features or external dependencies.

5.2 Secondary Objectives (Performance & Control)

The secondary objectives support the main goal by improving usability, structure, and learning value.

✓ **To ensure clear program structure using the Student class as the single data model**

Grouping all student-related data and operations inside the Student class demonstrates the single-responsibility principle and shows how OOP organises code into logical units.

✓ **To provide smooth operation through basic input handling**

The system should handle common input scenarios correctly, such as when the operator enters:

- A name that contains spaces (handled with getline)
- A roll number that does not exist in the system (handled with a found flag)
- A choice outside the valid menu range (handled with the default case)

✓ **To use simple, readable, and beginner-level C++ code**

The project intentionally avoids:

- File handling and binary streams
- Dynamic memory allocation with new/delete
- Pointer arithmetic
- Inheritance and polymorphism
- OS-dependent system calls

✓ **To encourage logical thinking and modular design**

Breaking the system into class methods (addDetails, updateDetails, displayDetails, getRollNumber) teaches students how to divide complex programs into manageable, testable units.

✓ **To make the system easily expandable for future improvements**

The code is kept clean so additional features such as file persistence, sorting, or a GUI can be added in future semesters without rewriting the core logic.

SYSTEM DESIGN & CODING

The Student Management System is a simple terminal-based application created using the C++ programming language. The design follows an object-oriented modular approach where data and related behaviour are encapsulated inside the Student class, while overall control flow is managed by the main() function. This makes the program easier to understand, debug, and maintain—especially for beginners.

6.1 System Architecture

The system is designed using an object-oriented architecture with a clear separation between the data model and the control layer.

Main Components:

6. Student Class (Data Model) — encapsulates private attributes and provides public methods to interact with them.
7. std::vector Database — a dynamically resizable container that stores all Student objects at runtime.
8. Input Processing — reads strings safely using getline() and integers using cin, with buffer clearing via cin.ignore().
9. Menu & Switch Controller — a do-while loop that continuously presents options and a switch statement that routes each choice to the correct operation.
10. Search & Comparison — iterates the vector and compares getRollNumber() return values to locate target records.

This modular structure makes the program easier to read, enhances clarity, and follows good OOP practices taught in the second semester.

6.2 Data Structures Used

The Student Management System uses simple data structures suitable for second-semester students:

1. Student Class Attributes (Private Members)

Used to store all information about a single student:

```
private:
    string name;        // Full name (supports spaces via getline)
    int    rollNumber;  // Unique identifier for searching
    int    semester;    // Current semester (1-8)
    float  sgpa;        // Semester Grade Point Average
```

2. std::vector<Student> — Dynamic Object List

A resizable array of Student objects, acting as the in-memory database:

```
vector<Student> database;
database.push_back(newStudent);      // Adds a record
database.erase(database.begin()+i);  // Removes a record
```

3. Integer Variables

Used for loop counters, storing user choice, and roll number comparisons.

6.3 Function Descriptions

This section explains the core methods of the Student class and the operations performed in main().

6.3.1 addDetails() — Input Module

Purpose: Collects all student information from the user and stores it in the object's private fields.

```
void addDetails() {
    cout << "Enter Student Name: ";
    cin.ignore();
    getline(cin, name);
    cout << "Enter Roll Number: "; cin >> rollNumber;
    cout << "Enter Semester: ";   cin >> semester;
    cout << "Enter SGPA: ";        cin >> sgpa;
}
```

cin.ignore() is called before getline() to clear the newline character left in the buffer by the previous cin >> choice, preventing the name field from being skipped.

6.3.2 updateDetails() — Modification Module

Purpose: Prompts the operator for new values and overwrites the existing name, semester, and SGPA. Roll Number is preserved as the unique key.

```
void updateDetails() {
    cout << "Enter New Student Name: ";
    cin.ignore();
    getline(cin, name);
    cout << "Enter New Semester: "; cin >> semester;
    cout << "Enter New SGPA: ";     cin >> sgpa;
    cout << "Student details updated successfully!\n";
}
```

6.3.3 displayDetails() — Output Module

Purpose: Prints a single student record in a formatted one-line layout. Declared const because it does not modify the object.

```
void displayDetails() const {
    cout << "Roll No: " << rollNumber
         << " | Name: " << name
         << " | Semester: " << semester
         << " | SGPA: " << sgpa << endl;
}
```

6.3.4 getRollNumber() — Search Accessor

Purpose: Returns the private rollNumber value to allow main() to compare it against a search query without directly accessing the private field.

```
int getRollNumber() const { return rollNumber; }
```

6.3.5 Main Menu Loop & Control Flow

Role: The main() function hosts an infinite do-while loop that continuously displays the menu. A switch statement routes each valid integer choice to the correct operation block. Entering 0 exits cleanly.

```
do {
    // Display menu options 1-5 and 0
    cin >> choice;
    switch (choice) {
        case 1: /* Add     */ break;
        case 2: /* View    */ break;
        case 3: /* Search  */ break;
        case 4: /* Update  */ break;
        case 5: /* Delete  */ break;
        case 0: /* Exit    */ break;
        default: cout << "Invalid choice. Try again.\n";
    }
} while (choice != 0);
```

PROGRAM OUTPUT / SAMPLE RUN

This section provides a sample execution of the Student Management System. It demonstrates how the menu appears to the operator, how input is taken, and how the system responds to each operation. The output shown below is based on actual terminal interaction.

7.1 Sample Terminal Interaction

```
TERMINAL OUTPUT
--- COMPLETE STUDENT SYSTEM ---
1. Add New Student
2. Display All Students
3. Search by Roll No
4. Update Student Details
5. Delete a Student
0. Exit
Enter your choice: 1

Enter Student Name: Krishna Kohli
Enter Roll Number: 101
Enter Semester: 2
Enter SGPA: 8.5
Student added successfully!

Enter your choice: 1
Enter Student Name: Rahul Sharma
Enter Roll Number: 102
Enter Semester: 2
Enter SGPA: 7.9
Student added successfully!

Enter your choice: 2

--- All Students ---
Roll No: 101 | Name: Krishna Kohli | Semester: 2 | SGPA: 8.5
Roll No: 102 | Name: Rahul Sharma  | Semester: 2 | SGPA: 7.9

Enter your choice: 3
Enter Roll Number to search: 101

Student Found:
Roll No: 101 | Name: Krishna Kohli | Semester: 2 | SGPA: 8.5

Enter your choice: 4
Enter Roll Number to update: 102

Current Details:
Roll No: 102 | Name: Rahul Sharma  | Semester: 2 | SGPA: 7.9

--- Enter New Details ---
Enter New Student Name: Rahul Sharma
Enter New Semester: 3
Enter New SGPA: 8.1
Student details updated successfully!

Enter your choice: 5
Enter Roll Number to delete: 102
Student record deleted successfully.

Enter your choice: 0
```

```
Exiting program...
```

Explanation of Sample Run:

- The program begins by displaying the numbered menu inside a do-while loop.
- Option 1 is used twice to add two student records to the vector.
- Option 2 iterates the vector and calls displayDetails() on each object.
- Option 3 searches by roll number using getRollNumber() and prints the matching record.
- Option 4 locates the target object and calls updateDetails() to overwrite its fields.
- Option 5 uses vector::erase() to permanently remove the record from the list.
- Option 0 breaks the do-while loop and exits the program cleanly.

FUTURE ENHANCEMENTS

Although the current version of the Student Management System is simple and fully functional, there are several ways it can be improved in the future. These enhancements can increase the system's durability, usability, and educational value. The following subtopics describe potential upgrades that can be implemented as the developer gains more programming experience.

8.1 Adding File Handling for Persistent Storage

The most immediate limitation of the current system is that all data is lost when the program exits, because records are stored only in RAM inside the vector. A future version can persist records to disk using C++ file streams:

- Use ofstream to write each student record to a .txt or .csv file when a record is added or modified.
- Use ifstream to read and reload all records from the file when the program starts.
- This allows data to survive across sessions without any database engine.

Adding file handling would also introduce students to the <fstream> library, an important topic in third-semester programming courses.

8.2 Implementing Inheritance and Polymorphism

A more advanced version of the system can introduce class hierarchies to model different categories of students. For example:

- A base class Person can hold common attributes such as name and roll number.
- Derived classes such as UGStudent and PGStudent can extend Person with specific fields like programme, thesis title, or research supervisor.
- Virtual functions can allow a single display() call to print the correct format for each student type.

This enhancement would teach inheritance, the is-a relationship, virtual method dispatch, and polymorphism—core topics of a second-semester OOP course.

8.3 Database Backend Integration

Instead of using a vector and a flat file, the system can be connected to a relational database for persistent, queryable storage:

- SQLite — a lightweight, file-based SQL engine that can be integrated into a C++ project with a single header.
- MySQL or MariaDB — a full client-server database suitable for multi-user deployments.
- Queries such as SELECT, INSERT, UPDATE, and DELETE can replace the manual vector iterations.

This upgrade would allow the system to handle thousands of records efficiently and introduce students to SQL, a skill essential in professional software development.

8.4 Graphical User Interface (GUI) Version

In the future, the Student Management System can be upgraded from a terminal-based program to a graphical interface using technologies such as:

- C++ with Qt — a cross-platform GUI framework that provides forms, tables, and dialog boxes.
- C++ with wxWidgets — a lightweight alternative for building native-looking desktop applications.
- Java Swing or JavaFX — if the project is ported to Java for a later course.
- Web Interface — a REST API backend in C++ with a browser-based front-end using HTML, CSS, and JavaScript.

A GUI would allow features like:

- Table view displaying all students with sortable columns
- Search bar for real-time filtering
- Modal forms for adding and editing records
- Confirmation dialogs before deletion

CONCLUSION

The Student Management System project successfully demonstrates how fundamental Object-Oriented Programming concepts in C++ can be combined to create an interactive and practical administrative application. Through this project, core topics such as classes, encapsulation, getter methods, std::vector, loops, conditional statements, and user input handling were applied in a real-world context.

The modular structure of the program—with data and behaviour encapsulated inside the Student class and control logic managed by main()—makes it easy to understand, maintain, and extend. The system runs smoothly in a terminal environment and fulfils its purpose of providing a beginner-friendly learning experience for second-semester OOP students.

The project also highlights the power of std::vector as a dynamic data structure: adding, removing, and iterating records requires no manual memory management, allowing the developer to focus entirely on the application logic rather than low-level pointer operations.

Overall, the project is simple yet effective and serves as a strong foundation for future improvements such as file-based persistence, class inheritance, database integration, or a full graphical user interface. It highlights the importance of applying theoretical OOP knowledge to real-world applications and builds confidence for more advanced programming tasks in upcoming semesters.

SUBMISSION

Submitted by:	Submitted To:
Name: Krishna Kohli **Course:** B.Tech CSE Core **Roll No:** GF202565408	Mr. Anurag Rana Sir **Submitted On:** 27 April 2026

© 2026 Krishna Kohli | B.Tech CSE Core | GF202565408 | Submitted to Mr. Anurag Rana Sir